

DPI343

Computer Organization and Assembly Language

Lecture 10.2

Interrupts

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

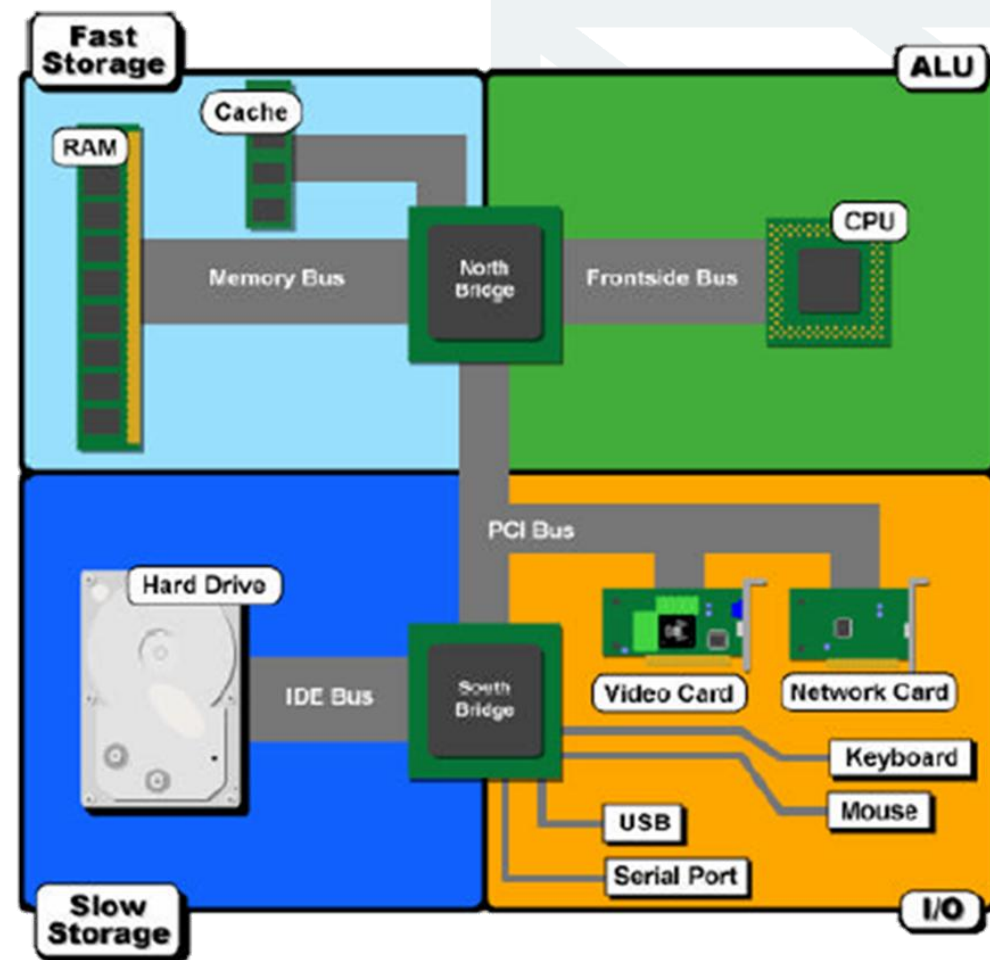
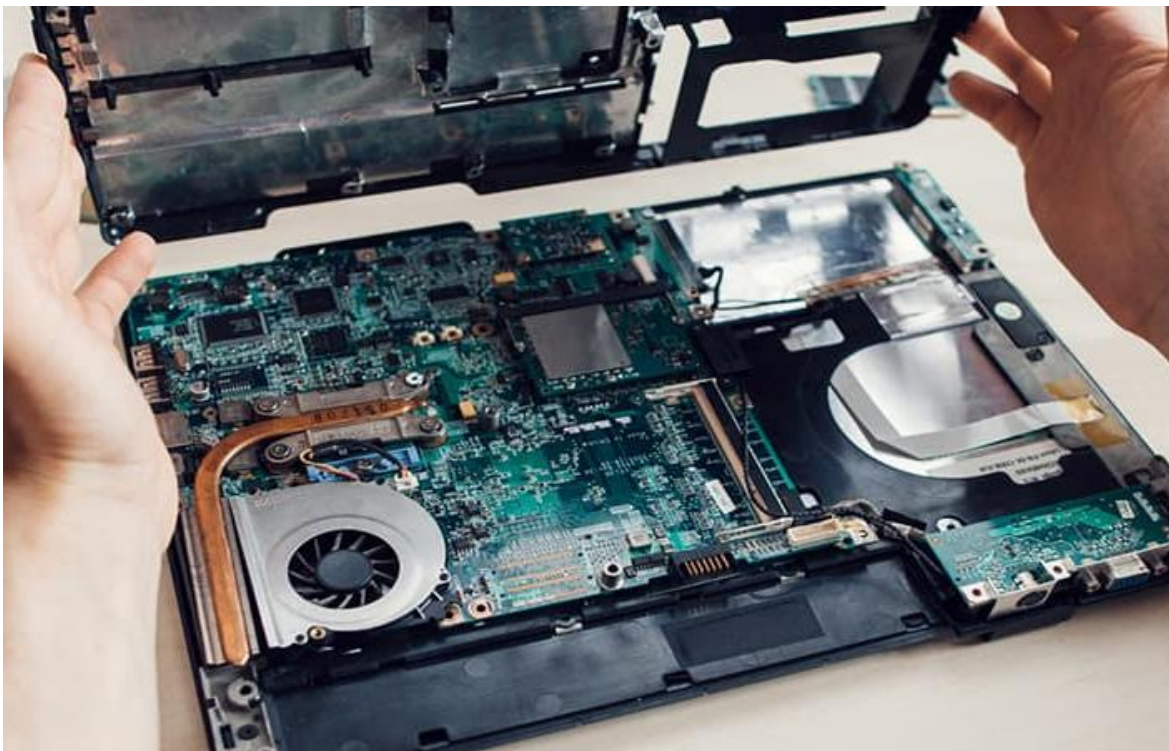
Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

Computer Components



Basic facts about interrupts

- An interrupt is a **signal to the CPU initiated by hardware or software** indicating an event that needs immediate attention
- CPU can interrupt the execution of the current process holding the CPU, saving its state, and executing a function called an **interrupt handler** (or an **interrupt service routine, ISR**)
- After the interrupt handler finishes, the **CPU resumes execution of the original process**
- Only a limited number of types of interrupts are defined
- Each type of interrupt has associated **priority level**
- When the CPU is executing an interrupt handler, it may be further interrupted by an interrupt source of higher priority

Interrupt sources

- Software interrupts – initiated by the instruction

INT n

where n is the interrupt number, e.g.:

```
mov ah, 0    ; BIOS function 0 (set video mode)
mov al, 3    ; function parameter (text video mode)
int 10h      ; transfer control to BIOS interrupt handler
```

Software interrupts cannot be masked (disabled).

- Hardware interrupts – initiated by devices connected to Programmable Interrupt Controller (PIC) or Advanced Programmable Interrupt Controller (APIC).
- Exceptions – faults (e.g., memory page fault), aborts (e.g., hardware error), traps (e.g., numeric overflow) are processor initiated interrupts.

Interrupt processing in Real address mode

Regardless of source (software or hardware), when an interrupt occurs:

1. The CPU pushes the flags register onto the stack
2. The CPU pushes the return address (current values of CS:IP) onto the stack (CS value first)
3. The CPU determines the cause of the interrupt (i.e., the interrupt number) and fetches the four byte interrupt vector from the memory address
0000:interrupt_number * 4
(i.e. segment 0 and offset value calculated as interrupt_number * 4)
4. The CPU loads CS and IP with segment and offset values from the interrupt vector, i.e. transfers control to the interrupt service routine (ISR)
5. ISR must save all registers it modifies
6. After interrupt processing the values of all modified CPU registers should be restored and the instruction **IRET** must be executed. Instruction **IRET** restores flags, CS, and IP registers from the stack

It all seems a bit like a procedure call...

ISR should be written very similarly to a procedure without parameters, only at the end of execution it executes IRET instead of RET.

Interrupt vectors in Real address mode

- Each 4 bytes of the first kilobyte 1024 bytes of the memory contains **interrupt vector** – address of interrupt processing routine's entry point

low order word high order word

offset	segment
--------	---------

- There are 256 entries in the interrupt vector table ($4 * 256 = 1024$)
- Address of an interrupt vector can be calculated as a number of interrupt multiplied by 4**
- When an interrupt occurs, the CPU calculates the location of the corresponding vector in memory and takes the far address from there to transfer control
- Example: memory doubleword with address 0000:0020 contains entry point address of interrupt service routine for interrupt number 8 (timer interrupt)

0000:0020

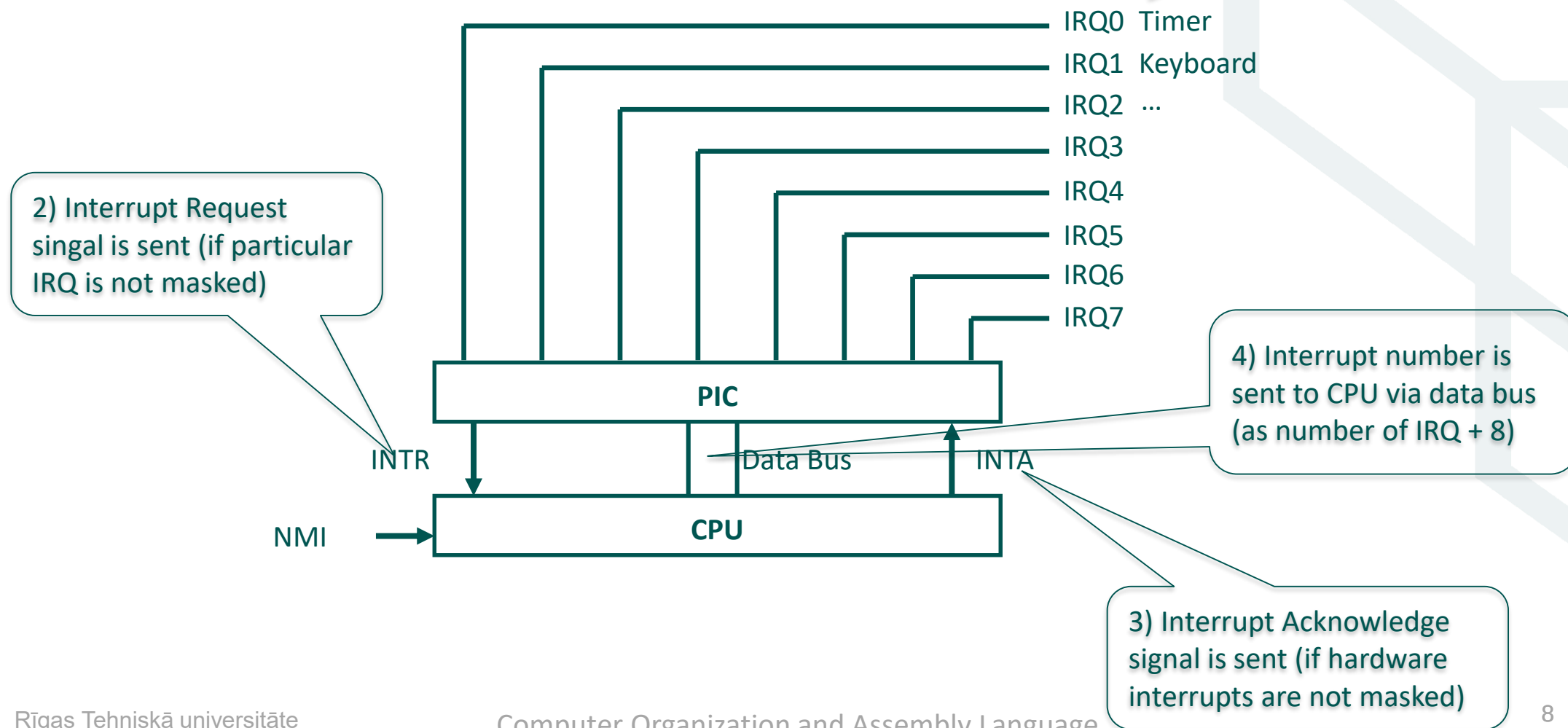
A5FE	00F0
------	------

entry point address is F000:FEA5

Intel's PIC and APIC

- **Programmable Interrupt Controller (PIC)** is a hardware component used to manage interrupts generated by hardware components
- Hardware components are connected to the PIC with **input interrupt request lines**
- When an interrupt occurs on any of PIC's input lines , it is processed by CPU
- The PIC can be programmed to enable or disable specific interrupt lines, and priorities are assigned to these lines
- Advanced Programmable Interrupt Controller (APIC) is a family of interrupt controllers used in multiprocessor systems

Hardware interrupts



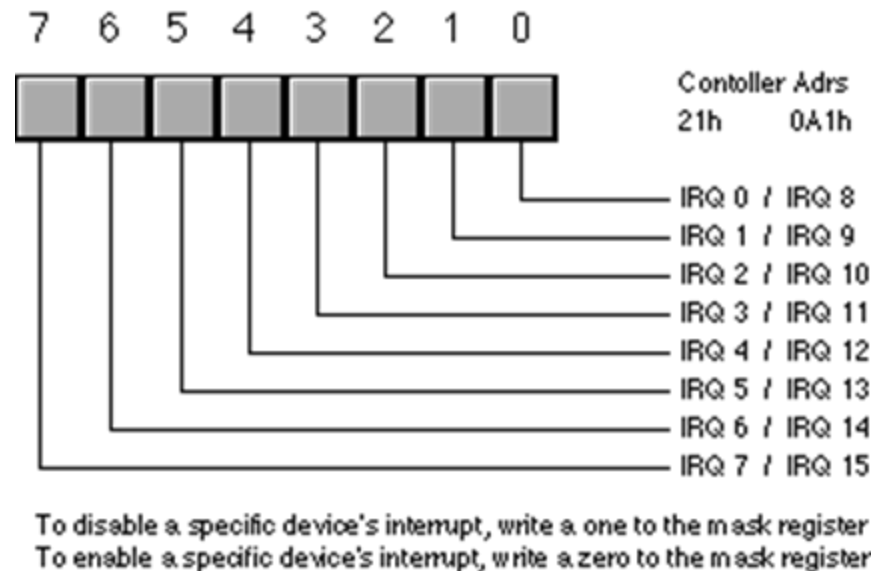
Processing hardware interrupts

- Each hardware interrupt has associated **priority level**. Interrupt initiated by IRQ0 has the highest priority, next IRQ1, etc.
- Upon entry into the hardware interrupt service routine the **CPU disables further hardware interrupts** by clearing the interrupt flag (IF) in the flags register
- To allow interrupts within a hardware interrupt service routine, you must explicitly **turn them on with an STI** instruction (set interrupt flag)
- Interrupt disable flag only affects hardware interrupts
- When hardware interrupt occurs, the PIC masks all further interrupts from that device **until it receives an end of interrupt signal (EOI)** from the interrupt service routine.
- **To send EOI** write the value 20h to the command register via the port 20h, e.g.:

```
mov AL, 20h  
out 20h, AL
```

Masking hardware interrupts

1. To disable all hardware interrupts, clear interrupt flag (IF) in the flags register by executing the instruction CLI.
2. Use PIC mask register to mask individual devices:



3. The mask register value can be set via port 21h

Example: Masking of keyboard interrupts

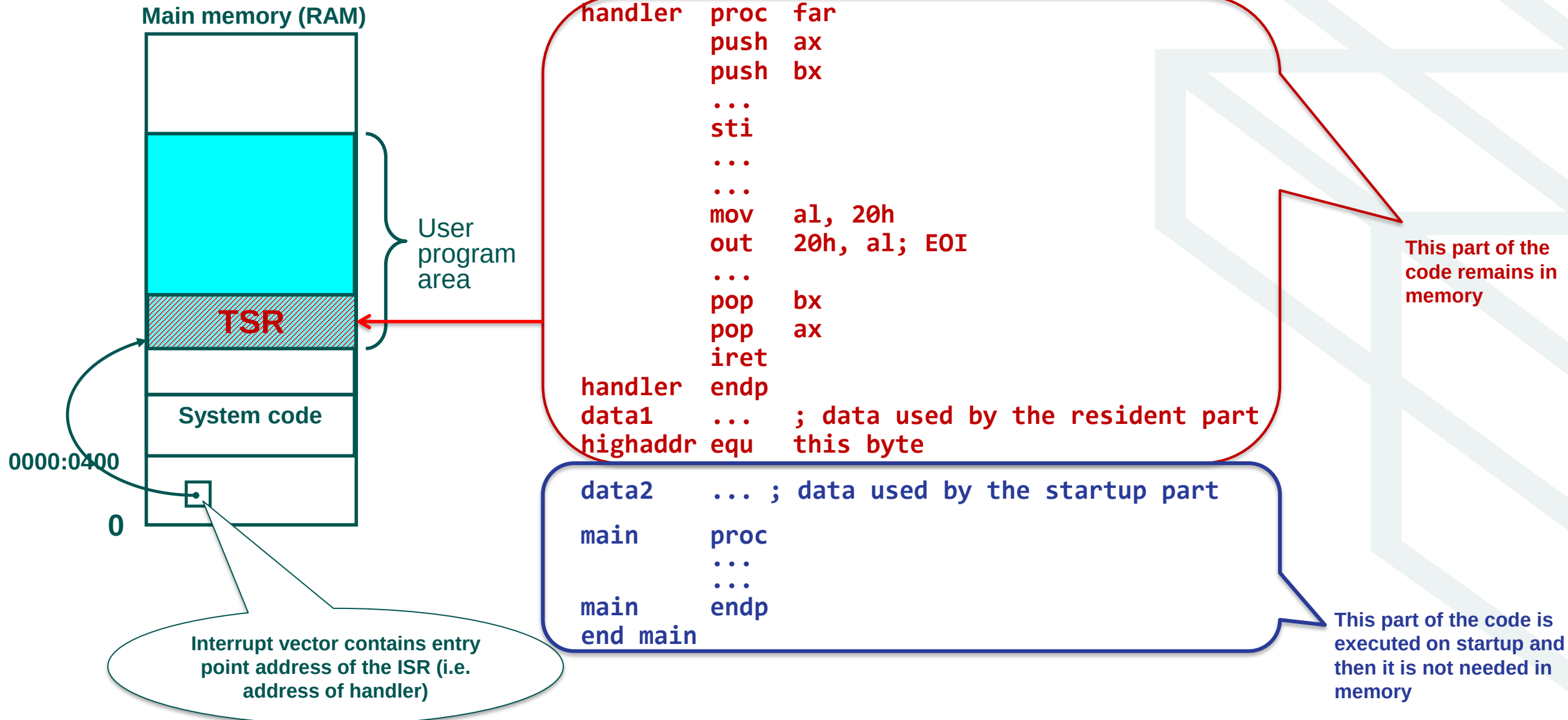
```
MOV AL, 00000010b  
OUT 21h, AL
```

Software interrupts cannot be masked!

Terminate and Stay Resident program (TSR)

- Operating system loads executable code of the program into memory, executes, terminates, and after then it can use the memory for the next program
- But the ISR must be in memory at the time of the interrupt
- **How can we keep our program code in memory?**
- Part of the program code can be left in memory after the program execution ends, i.e. part of the memory can not be returned to the system
- We will use this technique to permanently leave our own ISR in the computer's memory
- If we manage to do this, then all that remains is to write the address of our ISR's entry point into the interrupt vector and it will process the corresponding hardware interrupts

Terminate and Stay Resident ISR



System function to leave resident part in the memory

Terminate process and remain resident

AH = 31h

AL = exit code (returned to batch files)

DX = memory size in paragraphs to reserve

And execute int 21h

Memory size in paragraphs can be calculated from the start and end addresses.

When a program is loaded into memory, the system places a 256-byte PSP (Program Segment Prefix) before its code.

Program Segment Prefix – 256 bytes

handler proc

...

handler endp

...

highaddr:

Resident part to keep in memory

Example of the size calculation:

```
lea dx, highaddr
```

```
add dx, 10Fh      ; add the size of the PSP (+256) and round up to the paragraph (+15)
```

```
shr dx, 4         ; divide by 16
```

```
mov ax, 3100h     ; function 31h, return code 00h
```

```
int 21h           ; return to the system and keep memory
```

System functions to modify interrupt vectors

Get interrupt vector

AH = 35h

AL = number of the interrupt

And execute int 21h

Result: ES:BX = interrupt vector

```
mov ah, 35h
mov al, 8
int 21h
```

Set interrupt vector

AH = 25h

AL = number of the interrupt

DS:DX = new interrupt vector

And execute int 21h

```
mov ah, 25h
mov al, 8
mov dx, offset handler
int 21h
```

You can read and write these far addresses (i.e. segment and offset values) accessing vector memory directly with MOV instructions.

It is possible because we are working in Real address mode where memory protection is not enabled.

Thank You!

